A SIMD-Capable Execution Stage: Design Exploration and Characterisation

MEL G624 — Advanced VLSI Architectures, Semester I 2026–27 Assignment 1

Name	_____
ID No.	_____
Date of issue	24 October 2026
Date of submission	3 November 2026

1. Specification and interpretation

The task is to design the execution stage of a SIMD-capable datapath. It takes two 8-bit inputs `A` and `B`, a 2-bit function select `fn`, and a 1-bit size control, and produces two 8-bit outputs `C` and `D`.

Signal	Width	Meaning
`A`, `B`	8	operands, unsigned
`fn`	2	`00` add, `01` multiply, `10` equality, `11` reserved
`size`	1	`0` = one 8-bit operand per input, `1` = two packed 4-bit operands
`C`, `D`	8 each	results

The six specified behaviours are:

`size`	`fn`	`D`	`C`
0	add	`A + B`, carry-out discarded	0
0	mul	low 8 bits of `A × B`	high 8 bits
0	eq	`D[0] = (A == B)`, rest 0	0
1	add	`D[3:0] = A[3:0]+B[3:0]`, `D[7:4] = A[7:4]+B[7:4]`, each carry discarded	0
1	mul	`A[3:0] × B[3:0]`	`A[7:4] × B[7:4]`
1	eq	`D[0] = (A[3:0]==B[3:0])`, `D[4] = (A[7:4]==B[7:4])`, rest 0	0

Assumptions made where the specification is silent. The encoding `fn = 2'b11` is not defined; all four variants drive `C = D = 0` for it, and the testbench checks that. All arithmetic is

unsigned, as stated. Neither the 8-bit nor the 4-bit multiply can overflow its 16-bit and 8-bit result fields respectively (255×255 = 65025 and 15×15 = 225), so no saturation logic is required.

---

2. The central design question

The six behaviours above could be implemented by building six independent functional units and multiplexing their outputs. That solution is correct and it is what Section 4.1 calls Variant 1 — but it is the wrong answer, because in every cycle at least five of the six units compute a result that is discarded.

The useful observation is that **each 8-bit unit already contains the two 4-bit units it would otherwise sit beside.** The design work is in finding, for each operation, the minimum modification that lets one structure serve both modes.

2.1 Addition — breaking the carry chain

An 8-bit adder differs from two independent 4-bit adders only in the carry that crosses the nibble boundary. Gating that one wire is sufficient:

```
c_mid = c_out_of_low_nibble AND (NOT size)
```

With `size = 0` the carry propagates and the structure is an 8-bit adder; with `size = 1` the high nibble sees a zero carry-in and the two halves are independent. **The entire cost of SIMD support in the adder is one AND gate.**

In the speed-optimised variant the high nibble is evaluated speculatively for both carry-in values and selected afterwards, which moves this AND gate off the critical path — it then only drives a multiplexer select line, which settles while the two candidate sums are still being computed.

2.2 Multiplication — gating the cross products

This is where the real design content lies. Write the operands in terms of their nibbles, `A = 16·A_H + A_L` and `B = 16·B_H + B_L`. Then

```
A × B = 256·(A_H·B_H) + 16·(A_H·B_L + A_L·B_H) + (A_L·B_L)
                         \_________ cross terms _________/
```

The SIMD mode needs exactly `A_L·B_L` in `D` and `A_H·B_H` in `C`, and those are precisely the two terms that survive if the cross terms are forced to zero. What remains is

```
256·(A_H·B_H) + (A_L·B_L)
```

and because `A_L·B_L ≤ 225 < 256`, the two surviving terms occupy **disjoint bit ranges**. The 16-bit product therefore degenerates into a bit concatenation:

```
p = { A_H·B_H , A_L·B_L } = { C , D }
```

This is the key result of the assignment. A single 8×8 multiplier serves both modes with **no duplicated datapath and no output multiplexer at all** — both modes read their results off the very same product wires, `D = p[7:0]` and `C = p[15:8]`, in both cases. The only cost is that the 32 partial-product AND gates belonging to cross terms gain a third input, `~size`:

```
if (cross_term(i,j))  pp[i][j] = a[i] & b[j] & ~size;
else                  pp[i][j] = a[i] & b[j];
```

Because the gating happens at the leaves of the partial-product array, the reduction network behind it is completely unaware of the mode. Any multiplier microarchitecture — ripple array, Wallace, Dadda, Booth — can be dropped in unchanged, which is what makes Variants 3 and 4 a clean comparison.

2.3 Equality — one extra AND

Nibble-wise comparison gives `eq_lo` and `eq_hi` directly; the 8-bit result is `eq_lo AND eq_hi`. The 8-bit comparator and the SIMD comparator differ by one gate.

---

3. What was explored, and why

Four design points were built and measured. They are chosen to separate two independent questions — *does sharing help?* and *does hand-structuring help?* — rather than to pile up variations.

	Sharing	Arithmetic structure
V1 naive	none — duplicated units	inferred by the synthesiser
V2 shared	full	inferred by the synthesiser
V3 structural RCA	full	ripple-carry adder, shift-and-add array multiplier
V4 structural fast	full	carry-select adder, Dadda tree + Kogge-Stone final adder

V1→V2 isolates the benefit of sharing. V2→V3 isolates what is gained or lost by taking the arithmetic away from the synthesiser. V3→V4 isolates the benefit of logarithmic-depth structures over linear ones.

All four present an identical port list, so one testbench, one constraint file and one set of synthesis runs cover every design point. `simd_exec_top` selects between them with a `VARIANT` parameter.

---

4. Verification

A single golden model was transcribed directly from the specification text (not from any implementation) and all four variants were checked against it **exhaustively**:

```
2 size modes × 4 fn encodings × 256 A values × 256 B values = 524 288
vectors
524 288 vectors × 4 variants                                  = 2 097 152
comparisons
```

```
 vectors applied : 524288  (x4 variants = 2097152 comparisons)
 mismatches      : 0
 RESULT: *** ALL VARIANTS PASS ***
```

Exhaustive verification is affordable here because the input space is only 2^19 points including the mode bits, so there is no reason to settle for random or directed testing. The reserved `fn = 2'b11` encoding is included in the sweep.

A second, short testbench (`tb/tb_simd_wave.v`) applies 24 labelled stimuli covering every mode in 250 ns, for waveform inspection. It deliberately applies the same operand pair with `size = 0` and then `size = 1` so the carry chain and the cross products can be seen being broken:

```
add  0x3F+0x11 8b   A=3f B=11 fn=00 size=0 -> C=00 D=50
add  0x3F+0x11 4b   A=3f B=11 fn=00 size=1 -> C=00 D=40   <- carry
blocked
mul  0x33*0x22 8b   A=33 B=22 fn=01 size=0 -> C=06 D=c6
mul  0x33*0x22 4b   A=33 B=22 fn=01 size=1 -> C=06 D=06   <- cross terms
killed
```

---

5. Characterisation methodology

Synthesis is by **Yosys 0.33** with **ABC** technology mapping against the **Nangate 45 nm Open Cell Library** (typical corner). Area is reported in µm² and also as gate equivalents, where 1 GE is the area of a `NAND2_X1` (0.798 µm²). Delay is the ABC static-timing critical path through the mapped netlist, in picoseconds.

Every variant is mapped **twice**, under two different optimisation targets:

- **area-oriented** — ABC `map -a`
- **delay-oriented** — ABC `map -B 0.9` followed by buffering and gate up/down-sizing

Reporting both is what makes the comparison honest. A design that wins on area under an area-oriented tool may not be the right choice once the tool is told to chase frequency, and a single-target comparison hides that.

The flow is reproducible with `make report`.

---

6. Characterisation results

6.1 Complete execution stage

Area-oriented mapping

Variant	Cells	Area (µm²)	Gate equiv.	Delay (ps)	Fmax (MHz)	Area vs V1
V1 naive behavioural	589	583.1	731	1056	947	100%
V2 shared behavioural	490	469.8	589	1075	930	81%
V3 structural RCA + array	422	**423.7**	531	1550	645	73%
V4 structural CSel + Dadda	441	438.9	550	**880**	**1137**	75%

Delay-oriented mapping

Variant	Cells	Area (µm²)	Gate equiv.	Delay (ps)	Fmax (MHz)	Area vs V1
V1 naive behavioural	773	879.1	1102	535	1868	100%
V2 shared behavioural	639	**739.7**	927	564	1774	84%
V3 structural RCA + array	726	785.5	984	946	1057	89%
V4 structural CSel + Dadda	660	780.2	978	**494**	**2023**	89%

6.2 Area–delay product

The area–delay product (ADP) is the standard single-figure merit when neither area nor speed is individually constrained.

Variant	ADP, area-opt (µm²·ps)	ADP, delay-opt (µm²·ps)
V1 naive	615 955	470 598
V2 shared	504 992	416 923
V3 structural RCA	656 712	742 847
V4 structural fast	**386 144**	**385 643**

V4 has the best ADP under both mapping targets, by a margin of 23–35 % over the next best.

6.3 Where the area and the delay actually go

Block	Area-opt area (µm²)	Area-opt delay (ps)	Delay-opt area (µm²)	Delay-opt delay (ps)
SIMD adder, ripple carry	38.6	361	71.3	228
SIMD adder, carry select	45.5	284	88.3	175
SIMD equality comparator	17.3	102	68.9	91
SIMD multiplier, shift-add array	360.7	1460	736.8	922
SIMD multiplier, Dadda + Kogge-Stone	385.2	792	740.8	454

This table is the most useful one in the report. **The multiplier is the design.** It accounts for 85–88 % of the total cell area and well over 90 % of the critical path; the adder and the comparator are rounding errors beside it.

Two consequences follow directly:

- Replacing the shift-and-add array with a Dadda tree buys a **1.84× delay reduction for a 6.8 % area increase** (1460 → 792 ps for 360.7 → 385.2 µm²). This single change accounts for essentially all of V4's advantage over V3.
- Replacing the ripple-carry adder with a carry-select adder buys 1.27× on the adder itself, but the adder is not on the critical path of the block, so the effect at the top level is negligible. **Effort spent optimising the adder in this design is close to wasted.**

---

7. Analysis and recommendation

Sharing is free money. V1 → V2 removes 19 % of the area under area-oriented mapping and 16 % under delay-oriented mapping, for a delay cost of under 2 % and no change in behaviour. There is no scenario in which the duplicated-datapath design is preferable. The entire saving comes from the multiplier: V1 instantiates an 8×8 multiplier *and* two 4×4 multipliers where V2 instantiates one 8×8 multiplier with 32 gated AND gates.

Hand-structuring is not automatically a win. V3 is fully structural and is the smallest design under area-oriented mapping (423.7 µm²), but it is also by far the slowest (1550 ps) and it has the *worst* ADP of all four. Writing ripple-carry arithmetic by hand took work and produced a design that a synthesiser beats on every axis except raw area. The lesson is that structural RTL pays off only when the structure encodes something the tool will not infer — which is exactly what V4 does.

V4 dominates. Under both mapping targets V4 is faster than every other variant *and* smaller than V1 and V2. Its only concession is 3.6 % more area than V3, in exchange for being 1.76× faster. On a strict Pareto analysis the frontier contains just two points:

- **V3** — minimum area, if area is the only thing that matters
- **V4** — everything else

V1 and V2 are strictly dominated by V4 and should not be chosen under any objective.

Which solution is optimal for which need

If the need is…	Choose	Why
Absolute minimum area, speed irrelevant	**V3**	423.7 µm², 3.6 % smaller than V4
Maximum frequency	**V4**	494 ps / 2023 MHz, delay-oriented mapping
Best overall balance (the usual case)	**V4**	best ADP under both targets, 23–35 % ahead
Shortest design time	**V2**	behavioural, ~40 lines, still 19 % smaller than naive
FPGA target	**V2 or V4**	see caveat below

The gap between V3 and V4 on area is 15.2 µm² — about 19 gate equivalents, or roughly four NAND2s per percent of frequency given up. In almost any real design that trade is worth taking, which is why V4 is the recommended implementation.

Caveat on the implementation medium

These numbers are for a 45 nm standard-cell ASIC. On an FPGA the ranking would shift: dedicated carry chains make ripple-carry structures far cheaper relatively than they are in an ASIC, which would narrow or reverse the V3–V4 gap on the adder, and LUT-based mapping changes the

relative cost of the compressor tree. The Vivado flow in this repository (`docs/VIVADO.md`) is set up to measure exactly that; the comparison is left as a separate exercise because the two media are not meaningfully comparable on one axis.

---

8. Conclusions

1. The SIMD requirement is almost free if it is designed in rather than bolted on. One AND gate breaks the adder's carry chain; 32 three-input AND gates in place of two-input ones make a single 8×8 multiplier serve both modes.
2. The partial-product decomposition is the crux: zeroing the cross terms makes the 16-bit product collapse to `{A_H·B_H, A_L·B_L}`, which is exactly the packed SIMD result, with no output multiplexer required.
3. Resource sharing saves 19 % area at under 2 % delay cost — an unambiguous win.
4. The multiplier dominates both area (85–88 %) and delay (>90 %). Design effort belongs there and essentially nowhere else.
5. The structural carry-select + Dadda design (V4) is the recommended solution: best ADP under both mapping targets, fastest overall, and within 3.6 % of the minimum-area design.

---

9. Reproducing these results

```
make lint              # elaboration check
make sim               # short waveform testbench
make sim-exhaustive    # 524 288 vectors x 4 variants
make report            # synthesis + characterisation -> results/asic/
```

For the Vivado flow on Windows, see `docs/VIVADO.md`.

Appendix A — Verilog source listings